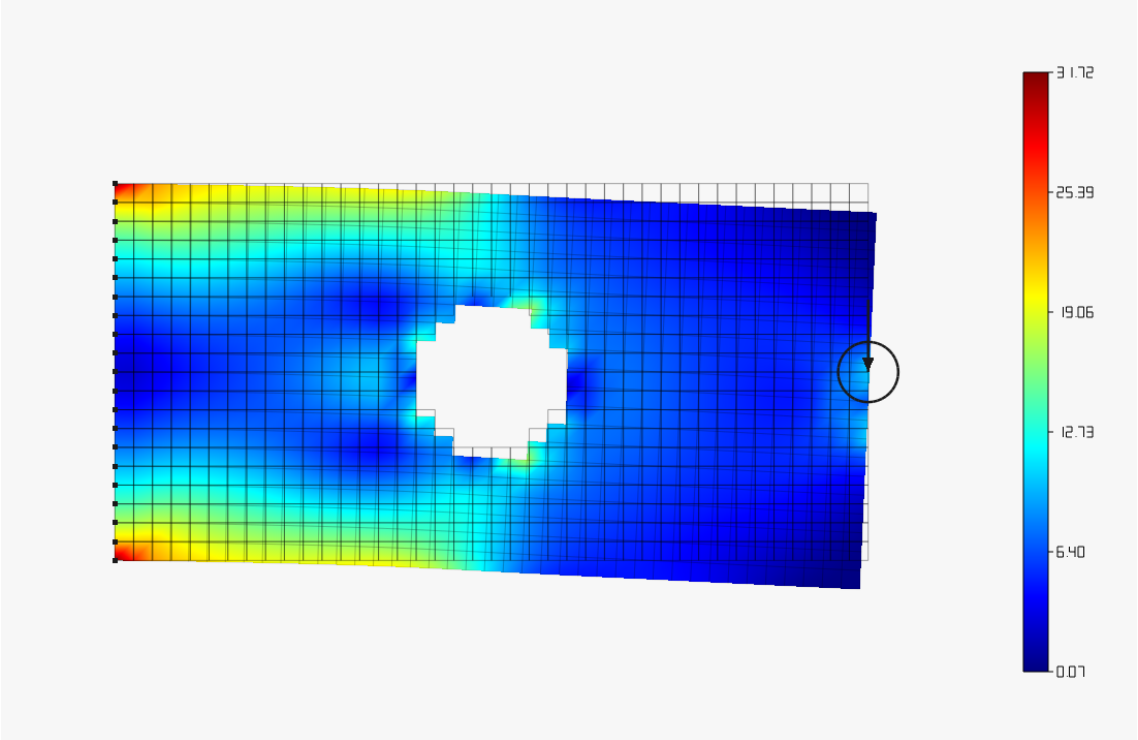
Interactive FEM Sheet

Build, run and understand `fem_sheet.cpp`

Q4 finite elements, thickness control, movable load, OpenGL stress contours



Default view: von Mises stress (MPa) in a 200 x 100 x 5 mm steel sheet with a hole, fixed on the left, loaded downward at the right edge.

1. What the program does

A thin steel sheet is meshed with bilinear quadrilateral (Q4) finite elements. You choose the thickness, the load magnitude, direction and position (by clicking or dragging on the sheet), the supports, plane stress or plane strain, and whether there is a hole. The program re-solves instantly and paints a **contour map** of the chosen stress quantity on the deformed sheet.

What is new compared with the first CST program

Feature	First program (fem2d.cpp)	This program (fem_sheet.cpp)
Element	3-node triangle, constant strain	4-node quadrilateral (Q4), strain varies inside the element
Integration	None needed (constant)	2 x 2 Gauss quadrature, Jacobian mapping
Cantilever accuracy	Tip deflection 17% too small	Within 0.02% of beam theory (self-test)
Geometry	Plain rectangle	Rectangle with optional hole (staircase boundary)
Thickness	Fixed in code	Changed live with arrow keys
Loads	Fixed, right edge	Any point, any direction, adjustable patch size; dragged with the mouse
Supports	Left edge	Cycle: left, left+right, bottom
Material model	Plane stress	Plane stress or plane strain (toggle)
Stress output	Von Mises per triangle	Six quantities, smoothed at nodes, smooth or banded contours, colour bar with values
Output	CSV files	Live OpenGL window, screenshots, headless modes

Files in the folder

File	Purpose
`fem_sheet.cpp`	The whole program in one file (about 680 lines).
`CMakeLists.txt`	Build recipe: finds Eigen, GLFW and OpenGL and produces the executable.
`FEM_Code_Explained.pdf`	Earlier guide to the basic CST code (concepts of FEM from scratch).

What I tested, and what I could not

Tested (Ubuntu 24.04): compiles with CMake without warnings; the self-test matches beam theory; the viewer renders correctly on a virtual display (software OpenGL), which produced every screenshot in this guide, including thickness, banding, field and load-position changes.
Not tested by me: live keyboard and mouse interaction in a real window, macOS and Windows builds. The input code is simple, but if a key or the mouse drag misbehaves on your machine, tell me what you see and I will fix it.

2. What we used (definitions)

Tool	What it is	How we use it
C++17	The programming language, compiled to a native executable.	Whole program; the std library gives vectors, math and file output.
Eigen 3	Header-only C++ linear algebra library (no separate binary to link).	Small dense matrices (B, D, Ke), sparse matrices and the sparse LDL^T solver.
GLFW 3	Small cross-platform library that opens a window, creates an OpenGL context and delivers keyboard/mouse events.	Window, key and mouse callbacks, buffer swap, window title (used as the status line).
OpenGL (fixed-function, 1.x)	The graphics API that draws triangles, lines and textures on the GPU (or a software renderer).	Draws triangles with a colour texture, line loops for the mesh, points for supports, quads for the colour bar.
CMake	Build-system generator. Reads CMakeLists.txt, finds the libraries, writes the build files.	Produces the executable in a few commands.
g++ / clang++	The C++ compiler.	Called by CMake (or directly with one command).

Why legacy OpenGL? The fixed-function calls (`glBegin`, `glVertex2d`) need no shaders and no extra loader library, so the code stays short and readable for learning. It works on Linux and Windows drivers. macOS marks it deprecated but still runs it (hence the GL_SILENCE_DEPRECATION flag). A modern shader-based version is a possible later step.

3. Build the executable

3.1 Ubuntu / Debian (also WSL2 on Windows 11)

```
sudo apt update
sudo apt install build-essential cmake libeigen3-dev libglfw3-dev libgl1-mesa-dev

cd fem_sheet                        # the folder containing fem_sheet.cpp
cmake -S . -B build                 # configure (finds Eigen, GLFW, OpenGL)
cmake --build build                 # compile
./build/fem_sheet --selftest        # quick check, no window
./build/fem_sheet                   # run the viewer
```

Expected self-test output (this is the real output from my build):

```
nodes=909 elements=800 dof=1818
tip deflection FEM (Q4): -0.200040  beam theory: -0.200000  ratio: 1.000
```

3.2 One-command alternative (no CMake)

```
g++ -O2 -std=c++17 fem_sheet.cpp -o fem_sheet -I/usr/include/eigen3 -lglfw -lGL
```

3.3 Other systems (not tested by me)

System	Commands
Fedora	`sudo dnf install gcc-c++ cmake eigen3-devel glfw-devel mesa-libGL-devel`
Arch	`sudo pacman -S base-devel cmake eigen glfw mesa`
macOS	`brew install cmake eigen glfw`, then the same three cmake commands. If CMake cannot find them: `cmake -S . -B build -DCMAKE_PREFIX_PATH=$(brew --prefix)`
Windows	Easiest is WSL2 on Windows 11 (graphical Linux apps work through WSLg) and the Ubuntu steps above. Native builds need MSYS2 or vcpkg and are not covered here.

3.4 Troubleshooting

Symptom	Fix
CMake: Could not find glfw3 / Eigen3	Install the dev packages listed above (names end in -dev or -devel).
"GLFW init failed" or "could not create window"	No display available (SSH without X forwarding, or a container). Run on the desktop session, use WSLg, or use the headless `--shot` mode with `xvfb-run`.
Window opens but stays blank	Update graphics drivers; on virtual machines enable 3D acceleration; try `LIBGL_ALWAYS_SOFTWARE=1 ./build/fem_sheet`.
Linker error: undefined reference to glfw / GL	The libraries must come after the source file in a manual g++ command (as shown in 3.2).
Binary copied to another machine will not start	That machine needs the GLFW runtime package (`libglfw3` on Ubuntu).

4. Using the viewer

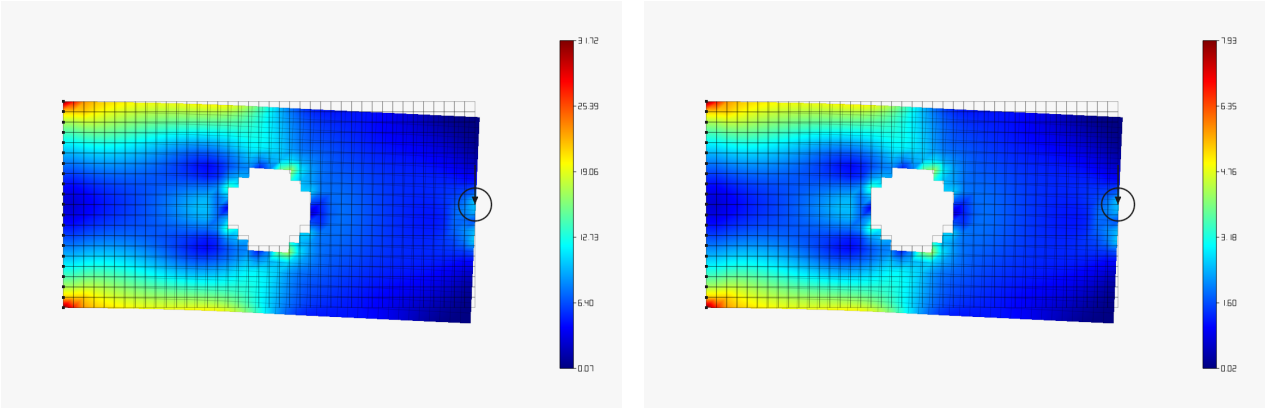
The window title is the status line: current stress quantity and its colour range in MPa, thickness, force, angle, plane stress or strain, supports, mesh size, maximum displacement and the deformation exaggeration factor.

Input	Action
Left mouse click / drag	Move the load point (the circle is the load patch, the arrow shows the force direction)
Q / E	Rotate the force direction by -15 / +15 degrees
Left / Right arrow	Decrease / increase force magnitude (x1.25 per press)
Up / Down arrow	Increase / decrease sheet thickness (x1.25 per press, range 0.2 to 100 mm)
, and .	Shrink / grow the load patch radius
1 2 3 4 5 6	von Mises, sigma_xx, sigma_yy, tau_xy, max principal, min principal
[and]	Decrease / increase the deformation exaggeration
- and =	Coarser / finer mesh (20x10, 40x20, 80x40, 120x60 cells)
H	Toggle the hole
B	Cycle supports: left edge, left + right edges, bottom edge
S	Switch plane stress / plane strain
C	Smooth colours / banded contours (12 bands)
K	Clip the colour range to 100%, 50%, 25%, 10% of max (reveals detail when a peak dominates)
M / U	Show or hide mesh lines / the undeformed outline
P	Save the current frame to shot.ppm
Esc	Quit

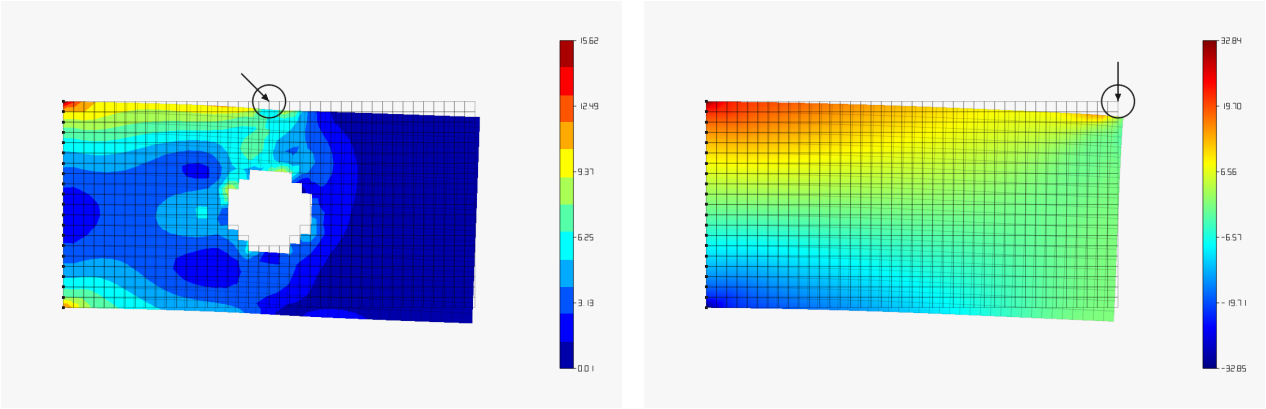
Start-up options (combine freely): `--thickness 10 --force 2000 --angle -45 --load 100 100 --field 2 --banded --nohole --mesh 2`. Use `--shot out.ppm` to render one frame to an image without interaction.

Experiments that teach something

- **Double the thickness.** Stresses fall to about half and displacements to about half, because the same force acts on twice the area. The screenshots below show 5 mm versus 20 mm: peak stress 31.7 MPa versus 7.9 MPa (a factor of 4 for 4x thickness).
- **Press 2 then 3.** sigma_xx is tension on top and compression on the bottom of the beam (bending). Compare with sigma_yy and tau_xy.
- **Press H.** The hole concentrates stress at its sides; remove it and the smooth bending pattern returns.
- **Press S.** Plane strain is stiffer (a thick body cannot contract through the thickness), so displacements shrink slightly.
- **Press = repeatedly.** Peak stresses near the fixed corners and near the load point keep growing as the mesh refines. These are stress singularities of the model (a point load and a sharp fixed corner), not a bug. Use K to look past them.



Left: thickness 5 mm, peak von Mises 31.72 MPa. Right: thickness 20 mm, peak 7.93 MPa (same 1000 N load).



Left: banded von Mises contours, load moved to the top edge at 45 degrees. Right: sigma_xx on a sheet without a hole (tension red on top, compression blue below).

5. The theory behind the upgrade

5.1 The Q4 (bilinear quadrilateral) element

The element has 4 nodes and 8 degrees of freedom (u and v at each corner). It is described on a perfect square with natural coordinates (xi, eta) from -1 to +1. The four shape functions are $N_i = (1/4)(1 + xi_i\ xi)(1 + eta_i\ eta)$, where (xi_i, eta_i) are the corner values (-1,-1), (1,-1), (1,1), (-1,1). Each N_i equals 1 at its own corner and 0 at the others.

Isoparametric mapping. The same shape functions interpolate both the geometry and the displacement: $x = \text{sum } N_i\ x_i$, $u = \text{sum } N_i\ u_i$. This lets a square in (xi, eta) map to any convex quadrilateral in (x, y). The **Jacobian** matrix J = [dx/dxi, dy/dxi; dx/deta, dy/deta] converts derivatives: derivatives with respect to x and y equal J^{-1} times derivatives with respect to xi and eta. The area element is dA = det(J) dxi deta. In the code: `dN` (2x4), `J = dN * xy`, `dNxy = J.inverse() * dN`.

Strain. The B matrix is built from dNxy exactly as for the triangle but is now **3 x 8** and depends on position inside the element, so strain is not constant. This lets the element represent bending far better than the CST.

Stiffness by Gauss quadrature. Ke = integral of $(t\ B^T\ D\ B)\ dA$ has no simple closed form, so it is evaluated numerically at 2 x 2 Gauss points (xi, eta = plus or minus 1/sqrt(3)), each with weight 1: **Ke = sum over 4 points of $t\ \det(J)\ B^T\ D\ B$** (8 x 8). For a rectangular element, 2 x 2 points integrate this exactly.

5.2 Material matrices

Case	D matrix (3 x 3)	When to use
Plane stress	$E/(1 - nu^2)$ times [1, nu, 0; nu, 1, 0; 0, 0, (1-nu)/2]	Thin sheet loaded in its plane (stress through thickness = 0)
Plane strain	E/((1+nu)(1-2nu)) times [1-nu, nu, 0; nu, 1-nu, 0; 0, 0, (1-2nu)/2]	Long thick body where strain through the thickness is 0

5.3 Thickness

Thickness t multiplies every element stiffness, so K is proportional to t. With the same load, displacements scale as 1/t and so do stresses (stress = force / area). Changing t forces a re-assembly and re-factorization; moving the load only needs a new right-hand side, which is why dragging the load is instant.

5.4 Stress recovery and smoothing

For each element, strain at each of its 4 corners is computed as B(corner) u_e, then stress as D times strain. A node shared by several elements receives several slightly different values; they are **averaged** at the node. Averaging the components (sigma_xx, sigma_yy, tau_xy) first and deriving the other quantities afterwards gives a continuous field, which is what makes the contour map smooth instead of blocky.

Quantity	Formula
Principal stresses	$\text{sigma}_{1,2} = (sxx + syy)/2 \text{ plus or minus } \text{sqrt}(((sxx - syy)/2)^2 + txy^2)$
Von Mises	$\text{sqrt}(0.5[(sxx - syy)^2 + (syy - szz)^2 + (szz - sxx)^2] + 3\ txy^2)$, with szz = 0 (plane stress) or nu(sxx + syy) (plane strain)

Accuracy note. Evaluating stress at the element corners and averaging is simple and gives good-looking plots, but it is slightly less accurate than extrapolating from the Gauss points. Use the colours to understand the pattern, and use mesh refinement before trusting a peak value.

6. Code map

Function / struct	Role
`Params`	All user-adjustable settings: geometry, material, thickness, load, supports, display options.
`buildMesh`	Makes a nx x ny grid of quads; skips cells whose centre lies inside the hole; renumbers only the nodes that are used (unused nodes would make K singular).
`bMatrix`	Shape-function derivatives, Jacobian and the 3 x 8 B matrix at any natural point.
`constitutive`	Builds D for plane stress or plane strain.
`assemble`	Loops over elements, 2 x 2 Gauss integration, scatters Ke into triplets, builds sparse K, applies supports by elimination, factorizes with SimplicialLDLT and keeps the factorization.
`solveLoad`	Builds the force vector (load spread equally over nodes inside the patch, or the nearest node), solves with the stored factorization and rebuilds the full displacement vector.
`recoverStress`	Corner stresses per element, averaged to nodes.
`updateField`	Derives the chosen scalar (von Mises, sigma_xx, ..., principal) at every node and its min and max.
`selftest`	Slender cantilever versus the beam formula $P L^3/(3 E I)$.
`makeTexture`	Creates a 256-pixel 1D colour texture (jet colours), smooth or 12 discrete bands.
`drawText`	Tiny 7-segment number renderer so the colour bar has labels without a font library.
`viewFor / drawScene`	World-to-pixel transform; draws the contour, mesh, supports, load arrow and colour bar.
`onKey / onMouse / onCursor`	Input callbacks; they only change parameters and set "dirty" flags.
`refresh`	Runs only the computation stages that the dirty flags require.

What recomputes when

You change	Recomputed
Load position, direction, magnitude, patch size	Right-hand side + solve + stress (factorization reused)
Thickness, supports, plane stress / strain	Assembly + factorization + solve + stress
Hole, mesh level	Mesh + everything below it
Stress quantity (1-6), clip, banding, mesh lines	Only the colouring or drawing

How the contour map is drawn

- Every quad is drawn as two triangles at its **deformed** position (original position + displacement x exaggeration).
- Each vertex gets one number, its nodal stress scaled to 0..1 against the colour-bar range, used as a **1D texture coordinate**.
- The graphics card interpolates that coordinate across the triangle and looks up the colour texture per pixel, which produces smooth contours. In banded mode the texture has 12 flat colour steps and nearest-neighbour lookup, giving contour-style bands.
- The deformation exaggeration is automatic: the largest displacement is scaled to about 8% of the sheet height. The factor is shown in the title; [and] adjust it.

7. Known limitations and next steps

- The hole boundary is a staircase of square cells, so stress concentrations at the hole are over-predicted locally. A body-fitted mesh (for example from Gmsh) fixes this.

- A point load and a fixed corner are singularities: their peak stress grows without bound as the mesh is refined. Interpret them qualitatively.
- Linear elastic material and small deformations only. Large deformation or plasticity need Newton iterations.
- Static only. Animation needs a mass matrix and time integration (Newmark).
- Natural extensions: 8-node or 9-node elements, Gauss-point stress extrapolation, importing meshes, a 3D tetrahedral version, GPU or multithreaded solvers (relevant to your HPC goals), and a shader-based renderer.